LAB 2

I. Objectives

After completing this lab, students will be able to:

Knowledge

Understanding the role and significance of frequent itemset mining and association rule mining in data mining. Grasp how to discover relationships between items that co-occur in transactional data. Specifically, students will understand:

1. The concepts of itemset, frequent itemset, and association rule.
2. The meaning of evaluation metrics in association rule mining such as Support and Confidence.
3. The working principles of two frequent itemset mining algorithms: Apriori and FP-Growth.
4. How to generate association rules from frequent itemsets and evaluate the quality of the resulting rules.

Skills

- Represent transactional data in an appropriate format for frequent itemset mining.
- Apply Apriori and FP-Growth algorithms to find frequent itemsets from transactional data.
- Compute support and confidence to evaluate association rules.
- Generate and select association rules that satisfy min_sup and min_conf thresholds.
- Analyze and interpret the meaning of the discovered association rules in the context of the data.

II. Theoretical Summary

1. Frequent itemsets and association rules

In data mining, a frequent itemset is a set of items that appear together in a transactional database with a frequency greater than or equal to a minimum threshold called minimum support.

The support of an itemset is the proportion of transactions that contain that itemset compared to the total number of transactions:

$$\text{support}(X) = \frac{\text{number of transactions containing } X}{\text{total number of transactions}}$$

An association rule has the form:

$$X \rightarrow Y$$

Where:

- X and Y are two itemsets
- $X \cap Y = \emptyset$

An association rule expresses that: if a transaction contains X, then it is likely to also contain Y.

The strength of a rule is measured by confidence:

$$\text{confidence}(X \rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

Example:

Suppose we have the following transactions:

TID	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer
T3	Milk, Diaper
T4	Bread, Milk, Diaper

If **min_sup = 50%**, the itemset **{Bread, Milk}** appears in 2 out of 4 transactions → **support = 50%** → it is a **frequent itemset**.

From this, we can generate the rule: **Bread → Milk**.

If 2 out of 3 transactions containing Bread also contain Milk, then: **confidence = 2/3 = 66.7%**

2. Apriori Algorithm

Apriori is an algorithm used to find frequent itemsets in transactional data. It is based on the idea: If an itemset is frequent, then all of its subsets must also be frequent.
Apriori works in steps:

1. Count the occurrences of each item to find frequent 1-itemsets.
2. Generate larger candidate itemsets (2-itemsets, 3-itemsets, …) from the frequent ones.
3. Compute the support of these candidates. R
4. emove itemsets that do not satisfy min_sup.
5. Repeat until no more frequent itemsets can be found.

Drawback**:** Apriori requires multiple scans of the database and generates many candidate itemsets.

3. FP-Growth Algorithm

FP-Growth is more efficient than Apriori because it does not generate candidate itemsets.

It works in two main steps:

Step 1: Build FP-Tree

- Scan the data to find frequent items.
- Sort items in descending order of frequency.
- Construct an FP-Tree to compress the transactional data.

Step 2: Mine the FP-Tree

- Generate conditional pattern bases from the FP-Tree.
- Build conditional FP-Trees.
- Extract frequent itemsets from these trees.

Thanks to the FP-Tree structure, FP-Growth reduces the number of database scans and is usually faster than Apriori for large datasets.

4. Generating Association Rules

After finding frequent itemsets, we can generate association rules to discover relationships between items.

5. Illustrative Example:

Given the following dataset with 6 transactions:

TID	Items
T001	A, B, D, E
T002	B, C, E
T003	A, B, D, E
T004	A, B, C, E
T005	A, B, C, D, E
T006	B, C, D

With **min_sup = 50%** and **min_conf = 70%**:

1. Find all frequent itemsets using **Apriori**.
2. Find all frequent itemsets using **FP-Growth**.
3. Generate all association rules from the frequent itemsets in (1).

Question 1:

The database consists of 6 transactions:

T001 = {A, B, D, E},

T002 = {B, C, E},

T003 = {A, B, D, E},

T004 = {A, B, C, E},

T005 = {A, B, C, D, E},

T006 = {B, C, D}.

With **min_sup = 50%**, this means an itemset must appear in **at least 3 transactions** to be considered frequent.

Step 1: Find Frequent 1-itemsets (L1). Count occurrences:

A: appears 4 times

B: appears 6 times

C: appears 4 times

D: appears 4 times

E: appears 5 times

All items appear ≥ 3 times ⇒ all are frequent. **L1 = {A, B, C, D, E}**

Step 2: From L1, we generate candidate 2-itemsets

From L1, we generate candidate 2-itemsets by combining items. The generated sets are: {AB}, {AC}, {AD}, {AE}, {BC}, {BD}, {BE}, {CD}, {CE}, {DE}.

After scanning the database to compute support for each set, we obtain:

AB: appears 4 times

AC: appears 2 times

AD: appears 3 times

AE: appears 4 times

BC: appears 4 times

BD: appears 3 times

BE: appears 5 times

CD: appears 2 times

CE: appears 3 times

DE: appears 3 times

With **min_sup = 3**, the itemsets **AC** and **CD** are removed because their support is less than 3.

Thus, the **frequent 2-itemsets** are:
{A, B}, {A, D}, {A, E}, {B, C}, {B, D}, {B, E}, {C, E}, {D, E}.

This set is denoted as **L2**.

Step 3: Generate candidate 3-itemsets

From L2, we generate candidate 3-itemsets by combining items. The possible 3-item combinations are: {A, B, C}, {A, B, D}, {A, B, E}, {A, C, D}, {A, C, E}, {A, D, E}, {B, C, D}, {B, C, E}, {B, D, E}, {C, D, E}.

The support results are:

ABC: appears 2 times (T004, T005)

ABD: appears 3 times (T001, T003, T005)

ABE: appears 4 times (T001, T003, T004, T005)

ACD: appears 1 time (T005)

ACE: appears 2 times (T004, T005)

ADE: appears 3 times (T001, T003, T005)

BCD: appears 2 times (T005, T006)

BCE: appears 3 times (T002, T004, T005)

BDE: appears 3 times (T001, T003, T005)

CDE: appears 1 time (T005)

Keeping only those with support ≥ 3, we obtain: **L3 = {A, B, D}, {A, B, E}, {A, D, E}, {B, C, E}, {B, D, E}**

Step 4: Generate candidate 4-itemsets

From L3, we generate candidate 4-itemsets. The only set satisfying the Apriori condition is: {A, B, D, E}. Checking the database, this itemset appears 3 times, so it satisfies min_sup. **L4 = {A, B, D, E}**

After this step, no further candidate 5-itemsets can be generated, so the algorithm stops.

Question 2:

Step 1: Count item frequencies

First, we scan the entire database to count item occurrences:

A: appears 4 times

B: appears 6 times

C: appears 4 times

D: appears 4 times

E: appears 5 times

Since all counts are ≥ 3, all items are **frequent items**.

Then, items are sorted in descending order of frequency:

Item	Support count
B	6
E	5
A	4
C	4
D	4

Step 2: Reorder transactions

Each transaction is reordered according to the above frequency order:

T001 → B, E, A, D
T002 → B, E, C
T003 → B, E, A, D
T004 → B, E, A, C
T005 → B, E, A, C, D
T006 → B, C, D

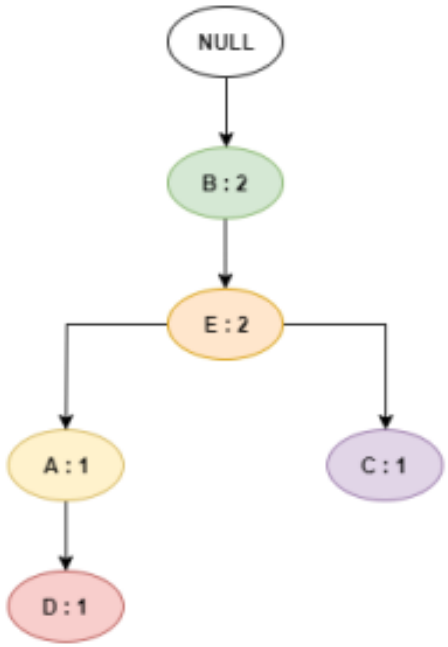
This ordering allows transactions with common prefixes to share paths in the FP-tree, reducing its size.

Step 3: Build FP-tree

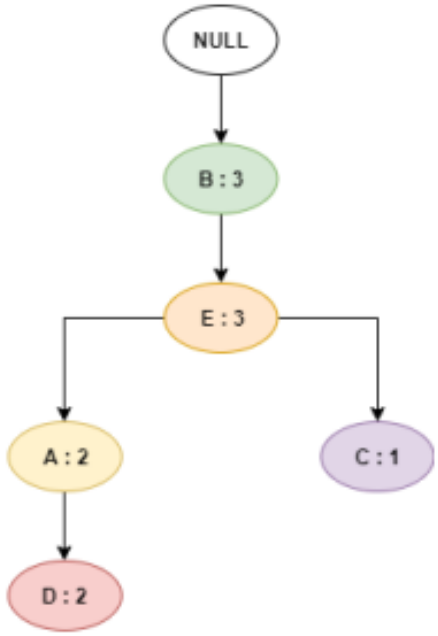
We start from the root node and insert transactions one by one:

The first transaction (B, E, A, D) creates a branch from the root.

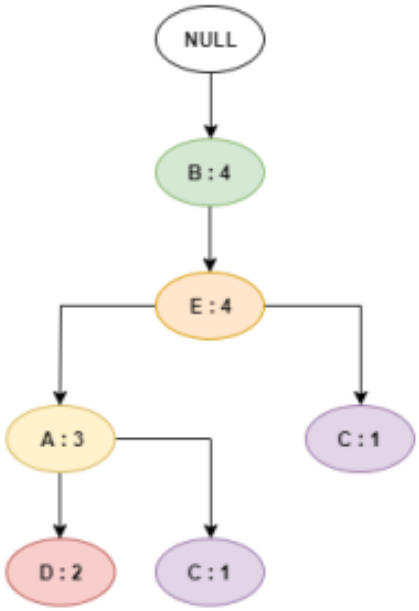
The second transaction (B, E, C) shares the prefix B and E, so we increment their counts and create a new branch C under E.



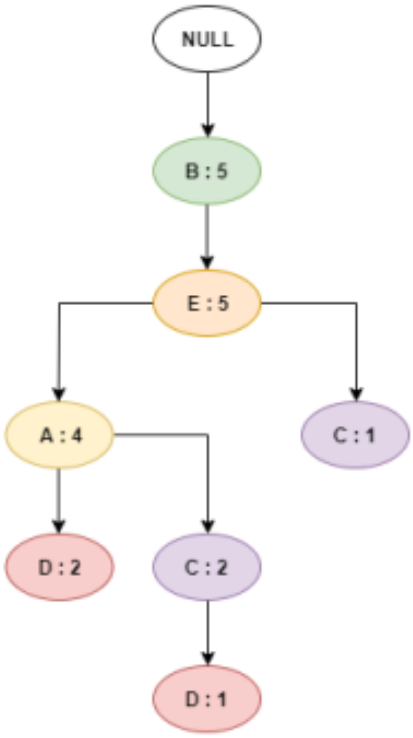
The third transaction (B, E, A, D) matches an existing branch, so we only increment counts of B, E, A, and D.



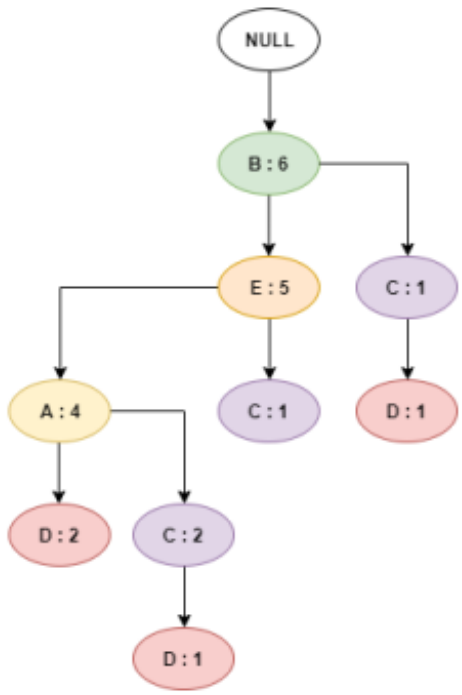
The fourth transaction (B, E, A, C) shares prefix B → E → A, then creates a new branch C under A.



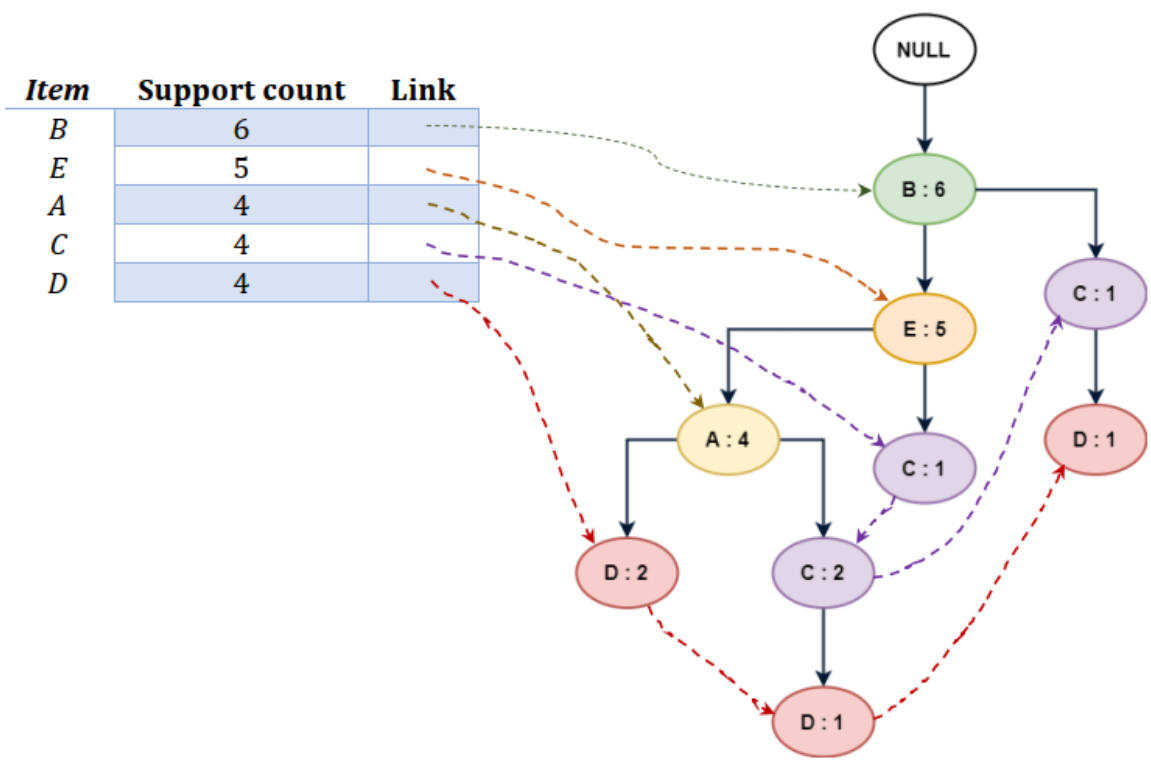
The fifth transaction (B, E, A, C, D) follows the path B → E → A → C and adds node D under C.



The last transaction (B, C, D) shares prefix B, then creates a new branch C → D.



After inserting all transactions, we obtain a complete **FP-tree** representing the entire transactional database.



Item	Support count	Link
B	6	
E	5	
A	4	
C	4	
D	4	

Step 4: Mining frequent itemsets from the FP-tree

After constructing the FP-tree, the FP-Growth algorithm does **not generate candidates** like Apriori but instead mines directly from the tree. The main idea is: For each item considered as a suffix, find all frequent patterns that contain that item.

Items are usually processed from least frequent to most frequent in the header table. In this case, the order can be: D, C, A, E, B

For each item, we perform three steps:

1. Find the conditional pattern base
2. Build the conditional FP-tree
3. Extract frequent itemsets containing that item

Conditional Pattern Bases:

Item	Conditional pattern base	Support count
D	B, E, A	2
	B, E, A, C	1
	B, C	1
C	B, E	1
	B, E, A	2
	B	1
A	B, E	4
E	B	5
B	Null	

4.1. Mining with item D

First, we select D as the suffix. We find all paths in the FP-tree ending at node D. For each D node, we take the path from the root to just before D (called the prefix path).

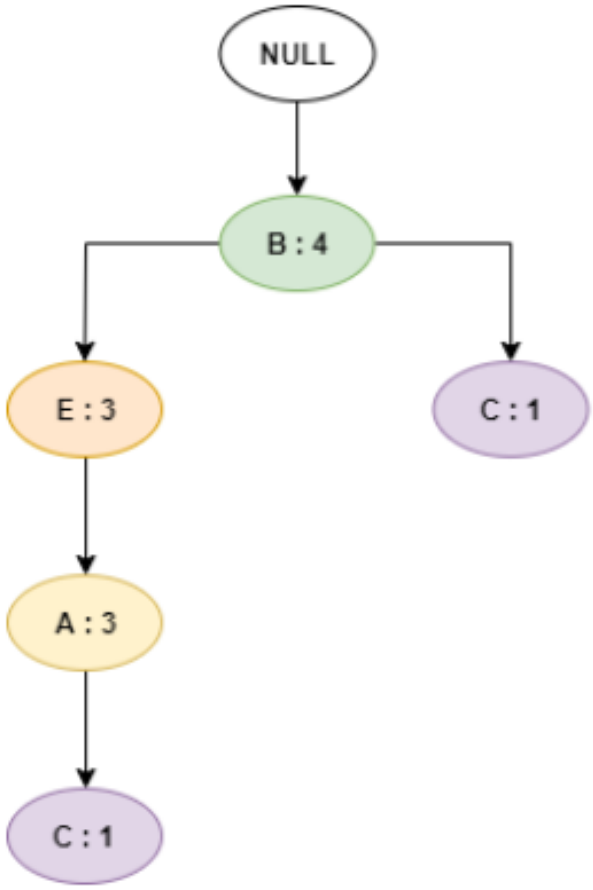
The paths leading to D are:

- B, E, A, D (count = 2)
- B, E, A, C, D (count = 1)
- B, C, D (count = 1)

Removing D gives the conditional pattern base of D:

- B, E, A : 2

- B, E, A, C : 1
- B, C : 1



Recount item frequencies:

- B: 2 + 1 + 1 = 4
- E: 2 + 1 = 3
- A: 2 + 1 = 3
- C: 1 + 1 = 2

Since **min_sup = 3**, item C is removed.

Frequent itemsets containing D:

- {D} (support = 4)
- {B, D} (support = 4)
- {E, D} (support = 3)

- {A, D} (support = 3)
- {B, E, D} (support = 3)
- {B, A, D} (support = 3)
- {A, E, D} (support = 3)
- {A, B, E, D} (support = 3)

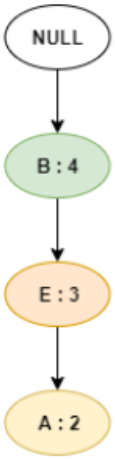
4.2. Mining with item C

Next, select C as the suffix. Paths ending at C:

- B, E, C (count = 1)
- B, E, A, C (count = 2)
- B, C (count = 1)

Conditional pattern base of C:

- B, E : 1
- B, E, A : 2
- B : 1



Recount:

- B: 1 + 2 + 1 = 4
- E: 1 + 2 = 3
- A: 2

Since **A < min_sup**, remove A.

Frequent itemsets containing C:

- support(C) = 4
- support(B, C) = 4
- support(C, E) = 3
- support(B, C, E) = 3

4.3. Mining with item A

Next, select **A** as the suffix. All A nodes lie on one main branch:

- B, E, A (count = 4)

Remove A. Conditional pattern base:

- B, E : 4

Recount:

- B appears 4 times
- E appears 4 times

Frequent itemsets containing A:

- support(A) = 4
- support(AB) = 4
- support(AE) = 4
- support(ABE) = 4

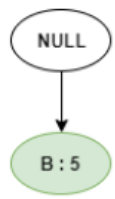
4.4. Mining with item E

Next, select **E** as the suffix. Path:

- B, E (count = 5)

Remove E, conditional pattern base:

- B : 5



Frequent itemsets containing E:

- support(E) = 5
- support(BE) = 5

4.5. Mining with item B

ally, select B as the suffix. Since B is the root-level item (no prefix path), its conditional pattern base is empty.

Frequent itemset:

support(B) = 6.

Question 3:

The frequent itemsets include:

1-itemset: $\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$;

2-itemset: $\{A, B\}, \{A, D\}, \{A, E\}, \{B, C\}, \{B, D\}, \{B, E\}, \{C, E\}, \{D, E\}$;

3-itemset: $\{A, B, D\}, \{A, B, E\}, \{A, D, E\}, \{B, C, E\}, \{B, D, E\}$;

4-itemset: $\{A, B, D, E\}$.

An association rule has the form:

$$X \rightarrow Y$$

with confidence:

$$conf(X \rightarrow Y) = \frac{support(X \cup Y)}{support(X)}$$

We only keep rules with **confidence ≥ 70%**.

Generate rules from 2-itemsets

From $\{A, B\}$, we have:

- $A \rightarrow B = 4/4 = 100\%$, keep.
- $B \rightarrow A = 4/6 = 66.7\%$, remove.

From $\{A, D\}$, we have:

- $A \rightarrow D = 3/4 = 75\%$, keep.
- $D \rightarrow A = 3/4 = 75\%$, keep.

From $\{A, E\}$, we have:

- $A \rightarrow E = 4/4 = 100\%$, keep.
- $E \rightarrow A = 4/5 = 80\%$, keep.

From $\{B, C\}$, we have:

- $B \rightarrow C = 4/6 = 66.7\%$, loại.
- $C \rightarrow B = 4/4 = 100\%$, keep.

From $\{B, D\}$, we have:

- $B \rightarrow D = 4/6 = 66.7\%$, remove.
- $D \rightarrow B = 4/4 = 100\%$, keep.

From $\{B, E\}$, we have:

- $B \rightarrow E = 5/6 = 83.3\%$, keep.
- $E \rightarrow B = 5/5 = 100\%$, keep.

From $\{C, E\}$, we have:

- $C \rightarrow E = 3/4 = 75\%$, keep.
- $E \rightarrow C = 3/5 = 60\%$, remove.

From $\{D, E\}$, we have:

- $D \rightarrow E = 3/4 = 75\%$, keep.
- $E \rightarrow D = 3/5 = 60\%$, remove.

Generate rules from 3-itemsets

From $\{A, B, D\}$,

We have:

- $AB \rightarrow D = 3/4 = 75\%$, keep.
- $AD \rightarrow B = 3/3 = 100\%$, keep.
- $BD \rightarrow A = 3/4 = 75\%$, keep.
- $A \rightarrow BD = 3/4 = 75\%$, keep.
- $D \rightarrow AB = 3/4 = 75\%$, keep.
- $B \rightarrow AD = 3/6 = 50\%$, remove.

From $\{A, B, E\}$,

We have:

- $AB \rightarrow E = 4/4 = 100\%$, keep.
- $AE \rightarrow B = 4/4 = 100\%$, keep.
- $BE \rightarrow A = 4/5 = 80\%$, keep.
- $A \rightarrow BE = 4/4 = 100\%$, keep.
- $E \rightarrow AB = 4/5 = 80\%$, keep.

From $\{A, D, E\}$,

We have:

- $AD \rightarrow E = 3/3 = 100\%$, keep.
- $AE \rightarrow D = 3/4 = 75\%$, keep.
- $DE \rightarrow A = 3/3 = 100\%$, keep.
- $A \rightarrow DE = 3/4 = 75\%$, keep.

- $D \rightarrow AE = 3/4 = 75\%$, keep.

From $\{B, C, E\}$,

We have:

- $BC \rightarrow E = 3/4 = 75\%$, keep.
- $CE \rightarrow B = 3/3 = 100\%$, keep.
- $C \rightarrow BE = 3/4 = 75\%$, keep.

From $\{B, D, E\}$,

We have:

- $BD \rightarrow E = 3/4 = 75\%$, keep.
- $DE \rightarrow B = 3/3 = 100\%$, keep.
- $D \rightarrow BE = 3/4 = 75\%$, keep.

Generate rules from 4-itemset $\{\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{D}, \boldsymbol{E}\}$

From this itemset, we consider all non-empty proper subsets.

The rules satisfying the confidence condition are:

- $A \rightarrow BDE = 3/4 = 75\%$
- $D \rightarrow ABE = 3/4 = 75\%$
- $AB \rightarrow DE = 3/4 = 75\%$
- $AD \rightarrow BE = 3/3 = 100\%$
- $AE \rightarrow BD = 3/4 = 75\%$
- $BD \rightarrow AE = 3/4 = 75\%$
- $DE \rightarrow AB = 3/3 = 100\%$
- $ABD \rightarrow E = 3/3 = 100\%$
- $ABE \rightarrow D = 3/4 = 75\%$
- $ADE \rightarrow B = 3/3 = 100\%$
- $BDE \rightarrow A = 3/3 = 100\%$

IV. Practice

Exercise 1

Given the file groceries.csv containing transaction data for one month at a store, use Python to perform the following tasks:

Question 1: Read the groceries.csv dataset and convert it into a list of transactions. Print the number of transactions and the number of items.

Hint:

```
with open("groceries.csv", "r") as f:
    for line in f:
        items = line.strip().split(",")
        transactions.append(items)
```

Question 2: Compute the number of transactions, the number of unique items, and the top 10 most frequent items.

Hint:

```
num_transactions = len(transactions)
all_items = [item for t in transactions for item in t]
unique_items = set(all_items)
counter = Counter(all_items)
```

Top 10 item: *counter.most_common(10)*

Question 3: Write code to compute the support of each 1-itemset.

Hint:

```
for item in unique_items:
    count = sum(1 for t in transactions if item in t)
    support_1[item] = count / num_transactions
```

Question 4: Find frequent 1-itemsets given a minimum support threshold.

Hint:

```
L1 = {item:sup for item,sup in support_1.items() if sup >= min_support}
```

Question 5: Write code to generate candidate 2-itemsets and find frequent 2-itemsets.

Hint:

```
items_L1 = list(L1.keys())
C2 = list(combinations(items_L1,2))
support_2 = {}
for pair in C2:
    count = sum(1 for t in transactions if set(pair).issubset(t))
    support_2[pair] = count/num_transactions
L2 = {pair:sup for pair,sup in support_2.items() if sup >= min_support}
```

Question 6: Implement the Apriori algorithm in Python. Write a function: ***apriori(transactions, min_support)***

Hint: Review the Apriori algorithm.

Question 7: Print intermediate steps C1, L1, C2, L2, ... of Apriori.

Hint:

```
L = apriori(transactions,0.01)
for i,l in enumerate(L):
    print(f"L{i+1}")
    for item,sup in l.items():
        print(item,sup)
    print()
```

Question 8: Use TransactionEncoder to one-hot encode the data.

Hint:

```
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array,columns=te.columns_)
```

Question 9: Use the mlxtend library to run Apriori.

Hint:

```
frequent_itemsets = apriori(df,
                            min_support=0.01,
                            use_colnames=True)
```

Question 10: Use the mlxtend library to run FP-Growth.

Hint:

```
fp_itemsets = fpgrowth(df,
                       min_support=0.01,
                       use_colnames=True)


print(fp_itemsets.head())
```

Question 11: Use the library to generate association rules from frequent itemsets.

Hint:

```
rules = association_rules(fp_itemsets,
                          metric="confidence",
                          min_threshold=0.2)
```

Question 12: Filter the top 10 rules by confidence.

Hint:

```
rules_sorted = rules.sort_values(by="confidence",ascending=False)
```

Question 13: Find rules whose consequent is "whole milk".

Hint:

```
rules_milk = rules[rules['consequents'].apply(lambda x: 'whole milk' in x)]
```

Question 14: Analyze the effect of min_support and min_confidence on the number of itemsets/rules.

Hint:

```
supports=[0.01,0.02,0.03,0.05]
counts=[]

for s in supports:
    itemsets = apriori(df,min_support=s,use_colnames=True)
    counts.append(len(itemsets))
```

Exercise 2

Given the file Online_Retail.csv containing transaction data for one month at a store, use Python to perform the following tasks:

Question 1: Read the file online_retail.xlsx and display basic information about the dataset. Clean the data by: Removing rows with missing CustomerID, Keeping only rows where: Quantity > 0 and UnitPrice > 0

Question 2: Convert the data into a transaction list, where each InvoiceNo is a transaction.

Hint**:** Use: groupby("InvoiceNo")

Question 3: Write a function to compute the support of an itemset.

$$Support(X) = \frac{\text{number of transactions containing } X}{\text{total number of transactions}}$$

Hint: Iterate through each transaction

Use: set(itemset).issubset(set(transaction))

Question 4: Compute the support of some specific itemsets.

Hint: Reuse the function from Question 3

Try with:

- 1-item
- 2-item

Question 5: Find frequent 1-itemsets with **min_support = 0.02**.

Hint:

Iterate over all items

Compute support

Filter by threshold

Question 6: Generate candidate 2-itemsets from frequent 1-itemsets.

Hint: Use combinations(items, 2) or union of itemsets

Question 7: Compute support and find frequent 2-itemsets.

Hint:

Reuse the support function

Filter by min_support

Question 8: Write a function to compute the confidence of the rule:

$$X \rightarrow Y$$

Write a function to compute the confidence of the rule:

Hint: Use the formula: support(X ∪ Y) / support(X)

Question 9: Implement the Apriori algorithm to find all frequent itemsets.

Question 10: Convert transaction data into one-hot encoding format.

Hint: Use TransactionEncoder. Output: DataFrame with 0/1 values

Question 11: Use mlxtend to run Apriori.

Hint: Import: from mlxtend.frequent_patterns import apriori

Input is a one-hot encoded DataFrame

Question 12: Chạy FP-Growth trên dataset.

Hint: Import: fpgrowth

Question 13: Generate association rules from frequent itemsets.

Hint**:** Use: association_rules()

Question 14: Filter strong rules with: confidence ≥ 0.7 and lift ≥ 1.2

Hint: Filter the DataFrame

Question 15: Plot a graph: min_support vs number of itemsets